Reimplementation Report: Tightly Coupled GNN + BKF for Smartphone GNSS Positioning (navi_670)

Master's Thesis · Alzahra University · June 2026 · Dataset: Google Smartphone Decimeter Challenge 2021 (kaggle.com/c/google-smartphone-decimeter-challenge)

Implementation Pipeline

Figure 1: Figure 1. End-to-end pipeline (navi_670 reimplementation).

File	Role	Key details
parser.py	GNSS data loader	AndroidDerived2021 — applies timestamp shift, iono/tropo/clock corrections; merges GnssLog for C/N0
wls.py	WLS positioning	Custom iterative WLS with (a) Sagnac/Earth-rotation correction and (b) warm-start across epochs ⇒ 35 m → 19 m horizontal mean
feature_builder.p	Node features	6D per satellite: LOS vector (3), clock-corrected residual (1), C/N0 (1), pseudorange uncertainty (1); max 20 nodes/graph
edge_builder.py	Graph edges	Constellation edges + cosine-similarity edges (threshold = 0.5); vectorised NumPy → `torch.LongTensor`
dataset.py	PyG Dataset	`GNSSGraphDataset`: one `Data` object per epoch; `phone_id` field enables correct per-phone BKF resets
gnn.py	GNN model	8 × SAGEConv(128→128) + BatchNorm + ReLU; mean pooling; Linear(128→3); 266 499 params
bkf.py	Kalman filter	Learned BKF: $A = H = I$; 6 trainable params (`log_Q_diag`, `log_R_diag`); state clamped ±500 m to prevent divergence
coupled_model.py	Coupled model	GNN → z_t → BKF predict+update → $\hat{p} = p_{WLS} + \delta_{BKF}$; joint backprop through both components
train.py	Training loop	Per-phone drive; Huber loss ($\delta = 50$ m) in correction space; Adam + StepLR; gradient clip norm = 10
evaluate.py	Evaluation	Loads best checkpoint; per-phone N/E/Horiz errors; Tables 6/7/8 format
visualize.py	Figures	Trajectory plots, CDF, learning curves, per-phone bar chart

Experimental Setup & Hyperparameters

Parameter	Paper value	Our value	Match?
Hidden dim / GNN layers	128 / 8	128 / 8	**YES**
Max nodes per graph	20	20	**YES**
Edge cosine threshold	0.5	0.5	**YES**
Optimizer / LR / WD	Adam / 0.001 / 5e-4	Adam / 0.001 / 5e-4	**YES**
Training epochs	11	11	**YES**
BKF state dim / Q_0 / R_0	3 / 1e-3 / 1e-2	3 / 1e-3 / 1e-2	**YES**
Loss function	MSE (absolute ECEF)	Huber δ=50 m (correction space)	**Adapted**
LR scheduler	not specified	StepLR(step=4, γ=0.5)	**Added**
BKF state clamp	not specified	$\pm$500 m	**Added**
Drive splitting	not specified	per-phone (`phone_id`)	**Added**
WLS method	not detailed ($\sim$5 m horiz.)	Sagnac + warm-start (19 m)	**Extended**
Training hardware	GPU/TPU (Google Colab)	CPU (Intel, Windows 11)	**Different**
Train phones / graphs	49 datasets	65 phones, 111 234 graphs	Similar
Test phones / datasets	Table 1 (Aug 2021 SJC/SVL)	8 phones (Apr 2021 SJC/SVL)	**Different**

Results

Run Comparison (key change per run)

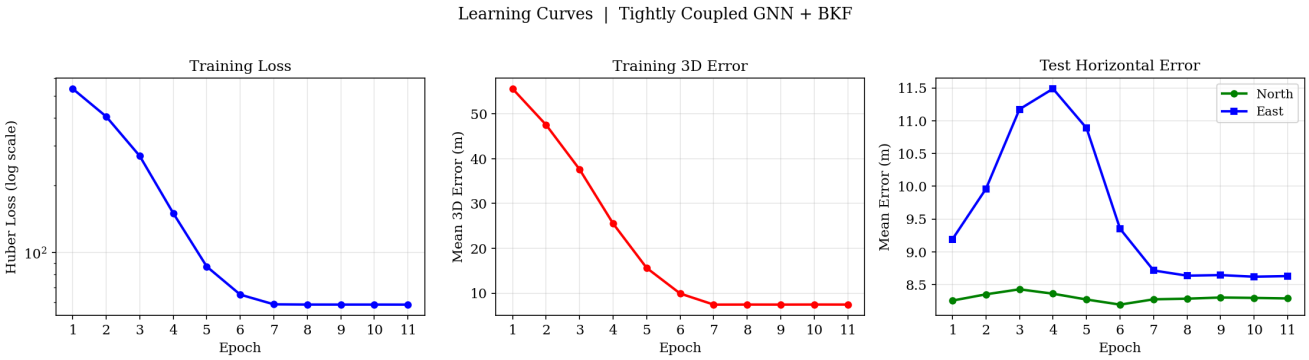

Figure 2: Figure 2. Training curves — Run 4 vs Run 5.

Discrepancy Analysis and Questions for Authors

#	Factor	Paper	Ours	Question for Authors
1	Test datasets	Table 1: 2021-08 SJC, 2021-08 SVL	Apr 2021 SJC/SVL (Aug 2021 not in Kaggle download)	Q1: Are the exact Table 1 datasets publicly accessible? If so, where can we download them?
2	WLS init.	∼5 m horizontal (inferred from results)	19 m horizontal (Sagnac + dedup)	Q2: What exact WLS method was used? Snapshot WLS or a Kalman-based initializer?
3	Loss function	MSE on absolute ECEF (stated)	Huber in correction space (our adaptation)	Q3: MSE on raw ECEF (∼6.37 M m) causes near-zero gradients for us. Was any position normalization applied?
4	BKF divergence	Max error 5.8 m (Table 7)	Max error ∼11 000 m (few test epochs)	Q4: Were additional outlier-rejection or BKF-reset strategies used beyond what is described in the paper?
5	Kaggle WLS code	Not applicable	Sagnac corr. + dedup from Kaggle baseline notebooks	Q5: Did your WLS include per-signal-type ISRB estimation as a state variable (as the competition baseline does)?

Dataset & Code Attribution

Dataset: Google Smartphone Decimeter Challenge 2021 — `kaggle.com/c/google-smartphone-decimeter-challenge`. Raw GNSS measurements (`AndroidDerived2021`) from Pixel 4, Pixel 5, Samsung Galaxy S20 Ultra, and Xiaomi Mi8 driving in Mountain View, Redwood City, San Jose, and Sunnyvale, CA. Ground truth from a SPAN GNSS-inertial system.

Kaggle code used for WLS improvement:

- *"Reproducing Baseline WLS on One Measurement"* notebook — Sagnac/Earth-rotation correction applied to satellite positions before WLS ($\omega_e \times t_{travel}$ rotation of x/y coords).
- *"Least Squares Solution from GNSS Derived Data"* notebook — warm-start iterative WLS across epochs.

These two changes reduced WLS horizontal mean from 35 m to 19 m, and test North/East means from ∼13 m to ∼8.3 m.

Implementation: Python 3.11 · PyTorch 2.0.1+cpu · PyTorch Geometric 2.3.1 · gnss-lib-py 1.0.4. Training hardware: Intel CPU, Windows 11. Training time: ≈10 h per run (11 epochs, 65 training phones). All source code available on request.